

# Whitebox Kriptografi

CEN429 Güvenli Programlama — Hafta 11

Dr. Öğr. Üyesi Uğur CORUH · 27.11.2026

## Bugünün planı (3 saat)

---

| Saat | Bölüm | Konu                                                                  |
|------|-------|-----------------------------------------------------------------------|
| 1    | 0–1   | Temel kavramlar · kara/gri/beyaz kutu · WBC saldırgan · naif çözümler |
| 2    | 2     | Tablo tabanlı WBC (Chow AES) adım adım · maliyet                      |
| 3    | 3–4   | Saldırı geçmişi · karşı önlemler · katmanlı yer · donanım · proje     |

## Bu hafta nereye oturuyor?

---

- **9. hafta:** kod gizleme — "gizleme anahtarı korumaz" dedik
- **Bu hafta (11):** peki anahtarı yazılımda **nasıl** koruruz? → whitebox
- **10. hafta:** kriptonun doğru kullanımı, anahtar yaşam döngüsü

Bugün "anahtar saldırganın elindeyken ne yaparız?" sorusunu işliyoruz.

# Öğrenme çıktısı

---

Bu hafta **ÖÇ.2 / ÖÇ.4** (kriptografi, güvenli iletişim) üstünedir.

Sonunda yapabileceğiniz:

- Kara/gri/beyaz kutu modellerini ayırmak
- Whitebox'ın **ne yaptığını ve neden yetmediğini** anlatmak
- Bir anahtar için doğru koruma kararı vermek

## Baştan bir uyarı

---

Bugüne kadar yayımlanmış **saf yazılım** whitebox tasarımlarının **hepsi kırıldı**.

O yüzden bugün "whitebox = güvenli anahtar" demeyeceğiz.

Diyeceğiz ki: whitebox, anahtar çıkarmayı **geciktiren bir katmandır** — ve tek başına yeterli değildir.

# 0. Temel kavramlar (sıfırdan)

# Şifreleme nedir?

---

- **Şifreleme (encryption):** okunur veriyi (**açık metin**), bir **anahtar** kullanarak okunamaz hale (**şifreli metin**) getirmek.
- **Çözme (decryption):** anahtarla geri açmak.

```
açık metin --(anahtar ile şifrele)--> şifreli metin  
şifreli metin --(anahtar ile çöz)--> açık metin
```

# Anahtar nedir?

---

- **Anahtar (key):** şifrelemeyi yöneten gizli sayı (bir bayt dizisi).
- Aynı algoritma + farklı anahtar = farklı sonuç.
- **Güvenlik anahtarın gizliliğine bağlıdır**, algoritmanın gizliliğine değil (Kerckhoffs ilkesi).

# Simetrik ve asimetrik

---

- **Simetrik:** şifreleme ve çözme **aynı** anahtar (ör. **AES**). Hızlı.
- **Asimetrik:** açık ve gizli anahtar çifti (ör. RSA). Yavaş ama anahtar dağıtımı kolay.

Bu hafta ağırlıklı **AES** (simetrik) üstünden gideceğiz.

## AES nedir? (yüksekten)

---

- **AES**: en yaygın simetrik şifreleme algoritması.
- 16 baytlık **bloklar** üzerinde çalışır.
- Veriyi **turlar** (rounds) halinde karıştırır; her turda: bayt değiştirme, satır/sütun karıştırma, anahtar ekleme.

Ayrıntı bu hafta sınavda sorulmayacak; **fikir** yeter.

## S-box nedir?

---

- **S-box (substitution box):** bir baytı başka bir baytla değiştiren **sabit bir tablo**.
- AES'in "bayt değiştirme" adımı budur.
- Örnek: `S-box[0x53] = 0xED` gibi 256 girişli bir eşleme.

Bu tabloyu aklınızda tutun; whitebox'ın kalbi budur.

## Arama tablosu (lookup table)

---

- **Arama tablosu:** "girdiye karşılık çıktı" veren dizi.
- $T[x]$  = x için önceden hesaplanmış sonuç.
- Hesap yapmak yerine **tablodan okursunuz**; hızlıdır.

Whitebox, hesabı tablolara gömer.

## XOR hatırlatma

---

- **XOR** (  $\wedge$  ): bitler farklıysa 1, aynıysa 0.
- $a \wedge b \wedge b == a \rightarrow$  geri alınabilir.
- AES'te "anahtar ekleme" adımı bir XOR'dur:  $\text{durum} \wedge \text{anahtar}$ .

## Bit güvenlik düzeyi

---

- "**n-bit güvenlik**" = kırmak için kabaca  $2^n$  deneme gerekir.
- AES-128 → 128 bit; bugün kırılmaz sayılır.
- Yüksek sayı = daha güçlü.

## Yan kanal (side channel) nedir?

---

- **Yan kanal:** algoritmanın matematiğini değil, çalışırken **sızdırdığı fiziksel/dolaylı bilgiyi** kullanan saldırı.
- Örnek: harcanan **zaman**, **güç** tüketimi, elektromanyetik yayım.

Kartlarda klasiktir; birazdan whitebox'ta yazılım karşılığını göreceğiz.

## DPA nedir?

---

- **DPA (Differential Power Analysis):** bir cihazın **güç tüketimi** ölçümlerine istatistik uygulayıp anahtarı çıkaran yan kanal saldırısı.
- Cihazın içine bakmaz; dışarıdan ölçer.

Bu fikri aklınızda tutun; whitebox'ın en güçlü saldırısı (DCA) bunun yazılım hâlidir.

## Bijeksiyon (birebir-örten eşleme)

---

- **Bijeksiyon:** her girdiyi tam bir çıktıya eşleyen, **tersi alınabilen** dönüşüm.
- Örnek:  $f(x) = x \wedge 0x5A$  bir bijeksiyondur (tersi yine kendisi).

Whitebox, tabloları **gizli bijeksiyonlarla** sarar.

## TEE ve güvenli öge (SE)

---

- **TEE (Trusted Execution Environment):** telefon işlemcisinin, işletim sisteminden **yalıtılmış** güvenli bölgesi.
- **Güvenli öge (SE):** anahtarları saklayan ayrı, kurcalamaya dirençli **donanım** çip.

Anahtar için en güçlü koruma bunlardır; whitebox bunların **olmadığı** durum içindir.

# HSM ve SoftHSM

---

- **HSM (Hardware Security Module):** sunucuda anahtarları saklayan/işleten özel donanım; anahtar **dışarı çıkmaz**.
- **SoftHSM:** HSM'in **yazılım benzetimi**; aynı arayüz (PKCS#11), ama donanım koruması yok (geliştirme/test için).

## Şimdi hazırız

---

Bildiğimiz terimler:

şifreleme/çözme · anahtar · simetrik/asimetrik · AES · S-box · arama tablosu · XOR · bit güvenlik · yan kanal · DPA · bijeksiyon · TEE/SE · HSM/SoftHSM

Şimdi asıl soruya: **anahtar saldırganın elindeyken ne yaparız?**

# 1. Üç saldırgan modeli

## Neden "model"?

---

Bir savunmayı tasarlamadan önce **saldırganın neyi görebildiğini** belirlemeliyiz.

Buna **saldırgan modeli** denir.

Üç model var: kara, gri, beyaz kutu.

# Kara kutu

---

- Saldırgan **yalnız girdi ve çıktıyı** görür.
- İçeriği (anahtar, ara değerler) göremez.
- Tipik ortam: **uzak sunucu** (kod sizde, saldırgan ağdan bağlanıyor).

AES/RSA'nın güvenlik kanıtları bunu varsayar.

## Gri kutu

---

- Saldırgan girdi/çıktıya ek olarak **yan kanalları** görür (zaman, güç, EM).
- İçeriği doğrudan okuyamaz ama **dolaylı** ölçer.
- Tipik ortam: elinde tuttuğu bir **akıllı kart** ya da gömülü cihaz.

## Beyaz kutu

---

- Saldırgan **her şeyi** görür: kod, bellek, tüm ara değerler.
- Üstelik **değiştirebilir**.
- Tipik ortam: **yazılım**, saldırganın kendi cihazında çalışıyor.

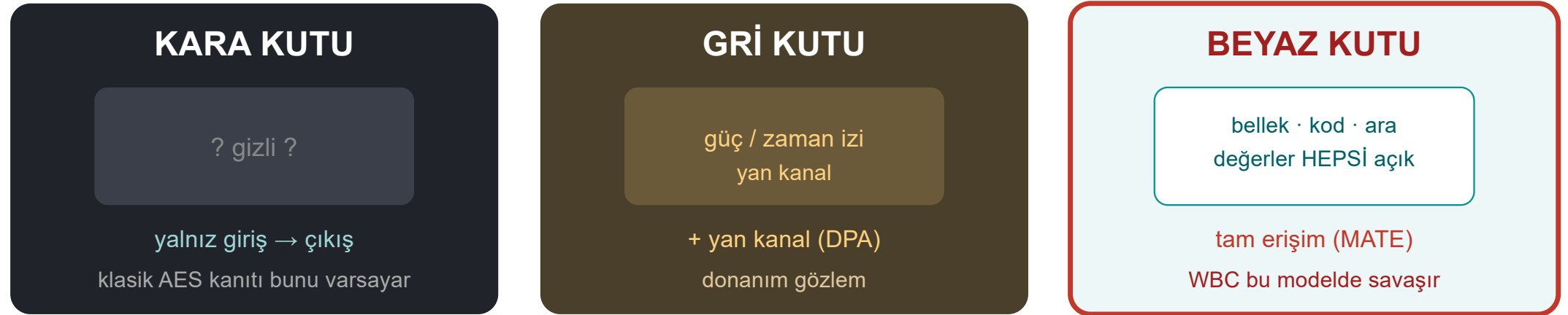
Bu, mobil/masaüstü uygulamanın gerçek durumudur.

## Üç model bir arada

| Model | Ne görür / yapar     | Ortam                 |
|-------|----------------------|-----------------------|
| Kara  | Girdi/çıktı          | Uzak sunucu           |
| Gri   | + yan kanal          | Akıllı kart           |
| Beyaz | Her şey + değiştirir | Yazılım, kendi cihazı |

Kripto **kara kutu** için tasarlandı; biz **beyaz kutudayız**.

# Üç saldırgan modeli — ne görüyor?



Görünürlük soldan sağa artar → WBC en zor modelde anahtar çıkarmayı geciktirir.

## Sorun burada

---

AES matematiksel olarak güçlü. Ama:

- Beyaz kutuda **anahtar bellekte** bir yerdedir
- Saldırgan belleği okuyabilir
- O zaman AES'in gücü ne işe yarar?

Güçlü kilit, anahtarı kapının yanına asarsanız işe yaramaz.

# Whitebox kriptografi (WBC) fikri

---

- **WBC:** şifrelemeyi, anahtar **hiçbir zaman açıkça bellekte olmayacak** biçimde gerçekleştirmek.
- Anahtar, önceden hesaplanmış **arama tablolarının içine** gömülür.
- Bellekte "işte anahtar bu" diyebileceğiniz bir bayt dizisi **yoktur**.

## WBC bir tanım değil, bir saldırgan modeli

---

- WBC, 2002'de Chow ve arkadaşları tarafından tanımlandı.
- Aslında bir **saldırgan modelidir**: "saldırgan her şeyi görüp değiştirebiliyorsa şifreleme nasıl yapılır?"

# WBC saldırgan neler yapabilir? (1)

---

## Gözlemler:

- O an işlenen komutlara erişir
- Algoritma akışını izler
- Kullanılan **belleği görür**

Yani programın gördüğü her ara değeri o da görür.

## WBC saldırgan neler yapabilir? (2)

---

### Değiştirir:

- Program belleğini kurcalar
- Algoritmanın **yalnız bir turunu** çalıştırır
- **if** koşullarını değiştirir
- Döngü sayaçlarını değiştirir
- **Hata enjekte eder** (bir değeri bozar)

## Kaynağın uyum maddesi

---

Teknik kılavuz bu bölümü şu gereksinimle açar:

"Kriptografik anahtarlar, gizlilik ve bütünlüğü korumak için **güvence altına alınmalı ve/veya gizlenmelidir.**"

## Ama hemen ekliyor

---

Kılavuz, bu saldırgan yeteneklerine karşı koruyanın **yalnız WBC olmadığını** söyler:

Uygulamayı ve varlıkları koruyan şey, **kod sağlamlaştırma (9. hafta) ve RASP (6. hafta)** yöntemleridir.

WBC ilk cümleden itibaren **tek başına** sunulmaz.

# Problem neden bu kadar zor?

---

Temel gerilim:

- Program şifrelemek için anahtara **erişmek** zorunda
- Saldırgan da programın gördüğü her şeyi görüyor
- Yani anahtar bir noktada **saldırganın da görebildiği** yerde

WBC bunu "anahtarı tablolara pişirerek" aşmaya çalışır.

## Bu gizlilik değil, maliyet

---

WBC gizliliği kriptodan gizlemeye **taşımaz**.

Matematiksel bir dönüşümle anahtarı tablolara **dağıtır**.

Amaç yine 9. haftadaki gibi: anahtarı geri çıkarmayı **çok daha pahalı** yapmak.

# Naif çözümler neden çöker?

## Neden çökenleri öğreniyoruz?

---

WBC'ye neden ihtiyaç olduğunu görmek için.

Her başarısız "kolay" çözüm, bir güvenli programlama **kuralına** dönüşür.

## Naif 1 · Anahtarı diziye göm

---

```
static const uint8_t k[16] = { 0x2B, 0x7E, ... }; /* KÖTÜ */
```

En kolay yol, en zayıf yol.

## Naif 1 · Nasıl kırılır?

---

1. `strings` / bayt taraması: 16 baytlık blok görünür
2. **Entropi taraması**: yüksek entropili blok "anahtar burada" der (Shamir–van Someren gözlemi)
3. Hata ayıklayıcı: şifreleme çağrısında bu bloğun kullanıldığını doğrular

**Süre:** dakikalar.

## Naif 1 · Kural

---

Düz gömülü anahtar bir anahtar koruması **değildir**.

Bu, 9. haftadaki "dize gizleme anahtar saklamaz" kuralının ta kendisi.

## Naif 2 · XOR ile "karıştır"

```
static const uint8_t k_gizli[16] = ...;    /* k ^ maske */  
/* çalışırken: k = k_gizli ^ maske */
```

"Anahtarı bir maskeyle XOR'layıp saklayayım."

## Naif 2 · Neden çöker?

---

- **Maske de** ikili dosyadadır
- Program **k** 'yı çözebiliyorsa, saldırgan da çözer
- Sadece bir adım fazladan; gerçek bir engel değil

Program neyi yapabiliyorsa, beyaz kutu saldırgan da yapar.

## Naif 3 tehdidi · Kod taşıma (code lifting)

---

- Diyelim WBC anahtarı tablolara gömdü.
- Saldırgan anahtarı **hiç çıkarmadan**, şifreleme yapan kod parçasını (tablolar + yorumlayıcı) **olduğu gibi kopyalar**.
- Kendi programında kullanır: anahtarı bilmeden işlevini **çalar**.

## Kod taşıma · Kural

---

WBC'nin "anahtarı gizledim" demesi yetmez.

Kod taşımaya karşı **cihaz/sürüm bağlama** gerekir (birazdan 3. bölüm): tablolar yalnız belirli bir cihazda/sürümde anlamlı olsun.

## Naif çözümler: özet

---

| Naif çözüm            | Kıran                                |
|-----------------------|--------------------------------------|
| Diziye gömülü anahtar | <code>strings</code> + entropi + gdb |
| XOR/whitening         | maske de içeride                     |
| (WBC bile)            | kod taşıma → cihaz bağlama gerekir   |

## Bölüm 1 — kısa sinama

---

1. Kara, gri, beyaz kutu farkı?
2. AES güçlüyken beyaz kutuda sorun ne?
3. Anahtarı diziye gömmek neden koruma değil?

## Bölüm 1 — cevaplar

---

1. **Kara:** yalnız girdi/çıktı · **Gri:** + yan kanallar (zaman/güç) · **Beyaz:** her şey + değiştirir.
2. Anahtar bir noktada **bellekte**; beyaz kutu saldırgan onu görür. AES'in gücü **kara kutu** varsayımına dayanır.
3. `strings` + **entropi** taraması + gdb ile dakikalar içinde bulunur; program onu kullanmak zorundadır.

## 2. Tablo tabanlı WBC (adım adım)

# Ne öğreneceğiz?

---

Chow ve arkadaşlarının 2002 AES tasarımı, tablo tabanlı WBC'nin klasığıdır.

- **Fikri** (nasıl çalışır)
- **Maliyeti** (tablo boyutu, hız)
- **Sınırı** (neden yeterli değil)

Sınavda matematik yok; **fikir ve sınır** var.

## Hatırlatma: bir AES turunun iki adımı

---

1. Durum baytını tur anahtarıyla **XOR**'la:  $x \oplus k$
2. Sonucu **S-box**'tan geçir:  $S\text{-box}[x \oplus k]$

Bu iki adım, sabit bir  $k$  için birleştirilebilir mi? Evet.

## Adım 1 · Anahtarı tabloya "pişir"

```
Normal:  çıktı = S-box[ x ^ k ]  
WBC:    T[x]  = S-box[ x ^ k ]   (256 girişli tablo)
```

Artık kodda **k** diye bir değişken **yok**. **k**, tablonun **içine** gömülü.

## Adım 1 · Ne kazandık?

---

- Bellekte `k` adında bir bayt dizisi yok
- Sadece `T` tablosu var (256 giriş)

Görünüşte anahtar kayboldu. Ama...

## Adım 1 · Neden tek başına güvensiz?

- $T$ , S-box'ın  $x^k$  ile **ötelenmiş** hali
- Saldırgan  $T$  'yi bilinen S-box ile karşılaştırır
- Öteleme miktarından  $k$  'yı **geri çıkarır**

Bu yüzden tabloları **kodlamak** gerekir.

## Adım 2 · Tabloları büyüt (T-box)

---

- Tek bayt eşlemesi yerine, S-box ile AES'in **karıştırma** adımını (MixColumns) birleştir
- Sonuç: 8 bit girip **32 bit** çıkaran daha büyük tablolar (**T-box**)
- Baytlar arası XOR'lar da küçük **XOR tabloları** (4 bit) olur

## Adım 2 · Sonuç

---

- Bütün tur, bir **arama tabloları ağına** dönüşür
- Hiçbir yerde açık bir aritmetik anahtar işlemi görünmez
- Ama tablolar hâlâ tek başına incelenebilir → kodlama şart

## Adım 3 · İç kodlamalar (internal encodings)

---

- Her tablonun **çıkışına** gizli bir bijeksiyon uygula
- Bir sonraki tablonun **girişinde** onun tersini uygula
- Ardışık kodlamalar birbirini **götürür** → sonuç doğru

Ama **ara değerler karışık** görünür.

## Adım 3 · Neyi engeller?

---

Bir tabloyu **tek başına** inceleyen saldırganı durdurmayı amaçlar.

Çünkü o tablonun girişi/çıkışı, gizli bir kodlamayla bozulmuştur.

## Adım 4 · Karıştırıcı bijeksiyonlar

---

- İç kodlamalara ek olarak,  $GF(2)$  üzerinde doğrusal **karıştırıcı matrisler** ( $8 \times 8$ ,  $32 \times 32$ )
- Bunlar bilgiyi tablolar arasında **yayar**
- Bir tablodan sızan bilgi tek başına anlamlı olmaz

*( $GF(2)$  = bitlerle, XOR aritmetiğiyle çalışan matematik; ayrıntı gerekmez.)*

## Adım 4 · Tablo tipleri

---

- Chow'un tasarımında bu kodlanmış/karıştırılmış tabloların **tipleri** vardır (literatürde I–IV).
- Her tip farklı bir rol üstlenir.
- Ders için: "tablolar tek tip değil, çeşitli" bilmek yeter.

## Adım 5 · Dış kodlamalar F ve G

- En dışta, tüm şifre iki gizli bijeksiyonla sarılır: **F** ve **G**
- Kaynak kılavuz: "whitebox AES işlemi F ve G adlı iki rastgele tekil-olmayan matrisle kapsülленir"
- Yani ağ artık saf AES değil:

```
Girdi --F-1--> [ kodlanmış tablolar ağı ] --G--> Çıktı  
= G ◦ AES ◦ F-1
```

## Adım 5 · Dış kodlama neden güçlü?

---

- Saldırganın gördüğü giriş/çıkış **standart AES'in giriş/çıkışı değil**
- Tek başına tablolara bakan pek çok saldırıyı (istatistiksel/cebirsel) çok zorlaştırır

## Adım 5 · Ama en büyük sınır da bu

---

$G \circ AES \circ F^{-1}$  standart AES değildir.

- Karşı taraf (sunucu/başka uç)  $F$  ve  $G$  'yi bilip geri almadan sonuç **anlamsız**
- Yani dış kodlamalı WBC yalnız **iki ucu da sizde** olan protokolde kullanılır
- EMV gibi **standart AES bekleyen** bir protokolde kullanılamayabilir

## Beş adım · özet

---

1. Anahtarı tabloya pişir
2. Tabloları büyüt (T-box + XOR tabloları)
3. İç kodlamalar
4. Karıştırıcı bijeksiyonlar
5. Dış kodlamalar (F, G)

Sonuç: anahtar açıkça yok; her şey kodlanmış tablolarda.

# WBC'nin maliyeti

## WBC bedava değil (1)

---

Anahtarı tablolara gömmenin bedeli: **kocaman tablolar**.

| Ölçüt         | Yaklaşık (mertebe)                  |
|---------------|-------------------------------------|
| Tablo boyutu  | Yüzlerce KB (~770 KB)               |
| Karşılaştırma | Std AES anahtarı: 16–32 <b>bayt</b> |

---

## WBC bedava değil (2)

---

| Ölçüt                    | Yaklaşık (mertebe)               |
|--------------------------|----------------------------------|
| Tur başına tablo okuması | Binlerce (~3000)                 |
| Hız                      | Std AES'e göre ~50× <b>yavaş</b> |

---

## Ölçü hissi: tablo boyutu hesabı

---

Örnek:  $288 \text{ tablo} \times 1 \text{ KB} \approx 294.912 \text{ bayt}$  ( $\sim 288 \text{ KB}$ ).

- Amaç kesin sayı değil, **mertebe**
- Bu yüzden WBC yalnız **seçili, kritik** işlemlere uygulanır
- Aynı mantık 9. haftadaki sanallaştırma (K-10) ile birebir

## Bölüm 2 — kısa sinama

---

1.  $T[x] = S\text{-box}[x \wedge k]$  tek başına neden güvensiz?
2. İç kodlama ile dış kodlama farkı?
3. Dış kodlamanın (F, G) en büyük sınırı ne?
4. WBC neden yalnız seçili işlemlere uygulanır?

## Bölüm 2 — cevaplar

---

1.  $T$ , bilinen S-box'ın  $x^k$  ile ötelenmiştir; iki tablo karşılaştırılıp **k geri çıkar**.
2. **İç**: tablolar arası ara değerleri karıştırır · **Dış (F,G)**: tüm şifreyi sarar (  $G \circ \text{AES} \circ F^{-1}$  ).
3. Artık **standart AES değildir**; karşı taraf F/G'yi bilmeli — EMV gibi standartlarda kullanılamayabilir.
4. Tablolar yüzlerce KB +  $\sim 50\times$  yavaş; maliyet yüzünden yalnız **küçük/kritik** işlemler.

## 3. Saldırı geçmişi ve karşı önlemler

## Neden geçmişe bakıyoruz?

---

Bir korumaya ne kadar güveneceğinizi, onun **kırılma geçmişi** söyler.

Her satır, savunmacı için bir **çıkarım**dır.

## 2002 · Doğuş

---

- Chow ve arkadaşları WB-AES ve WB-DES tasarımlarını yayımlar.
- Tablo tabanlı yaklaşım doğar.

**Çıkarım:** fikir umut verici; ama sınanmadı.

## 2004 · BGE saldırısı

---

- **BGE** (Billet–Gilbert–Ech–Chatbi): Chow AES'i pratik sürede kırar ( $\sim 2^{30}$  işlem).
- İç kodlamaları çözer.

**Çıkarım:** iç kodlamalar tek başına **yetmez**.

## 2007 · WB-DES düşer

---

- Goubin, Wyseur ve diğerleri WB-DES ve harici kodlamalı türevlerini kırar.

**Çıkarım:** DES tabanlı WBC bırakılır.

## 2010'lar · "Daha çok tablo" denemesi

---

- Karroumi'nin "çift AES" güçlendirmesi de kırılır ( $\sim 2^{22}$ ).

**Çıkarım:** daha çok tablo = daha çok güç **değildir**.

## 2015 · DFA (hata enjeksiyonu)

---

- **DFA (Differential Fault Analysis):** çalışırken bir değere **hata enjekte edip** bozulan çıktıdan anahtarı geri hesaplar.
- "Unboxing the White-Box" (Black Hat EU).

**Çıkarım:** tasarımın iç matematiğini **bilmeden** de kırılır.

## 2016 · DCA (en önemlisi)

---

- **DCA (Differential Computation Analysis):** çalışırken üretilen ara değerlerin **izini** toplar, DPA benzeri istatistikle anahtarı çıkarır.
- İç kodlamaları çoğu zaman **görmezden gelir**.

**Çıkarım:** klasik yan kanal savunmaları WBC için de gerekli.

## 2017–2024 · WhibOx yarışmaları

---

- Akademi ve endüstri tasarımlarını yarışmaya gönderir.
- Gönderilen tasarımların **tamamı kırıldı**.

**Çıkarım (bugün):** kırılmadan kalan yayımlanmış saf-yazılım WBC **yok**.

## Kronoloji · özet

| Yıl      | Olay             | Çıkarım                   |
|----------|------------------|---------------------------|
| 2002     | Chow WB-AES/DES  | Doğuş                     |
| 2004     | BGE              | İç kodlama yetmez         |
| 2007     | WB-DES kırıldı   | DES bırakılır             |
| 2010'lar | Karroumi kırıldı | Çok tablo $\neq$ güç      |
| 2015     | DFA              | Matematiği bilmeden       |
| 2016     | DCA              | Yan kanal savunması gerek |
| 2017–24  | WhibOx           | Hepsi kırıldı             |

## DCA ve DFA neden yeni tasarımları da vurur?

---

Çünkü ikisi de tasarımın **gizli matematiğini çözmeye çalışmaz**:

- **DCA**: kutuyu açmaz; çalışırken **iz** toplar, istatistikle anahtarı bulur
- **DFA**: çalışırken **hata enjekte eder**, bozuk çıktıdan hesaplar

Genel ve **taşınabilir** oldukları için her tasarımı tehdit ederler.

## Peki ne yapacağız? (karşı önlemler)

---

Araştırma durmadı; ama hiçbir önlem "artık güvenli" demez.

Hepsi **maliyeti artırır** ve **birlikte** kullanılır. Sırayla:

## Karşı önlem 1 · Doğrusal olmayan maskeleye

---

- DCA'nın dayandığı **doğrusal korelasyonu** bozmaya çalışır (ör. Biryukov–Udovenko 2018).
- İstatistiksel saldırıyı zorlaştırır.

## Karşı önlem 2 · Dış kodlama (F, G)

---

- DCA/DFA/BGE'yi zorlaştırır.
- **Ama** gördüğümüz sınır: standart AES uyumluluğunu kırar.

## Karşı önlem 3 · Katmanlı savunma

---

- WBC'nin çevresine 9. haftanın **gizleme** kuralları
- vi. haftanın **RASP**'ı (anti-debug, kurcalama/bütünlük denetimi)
- Amaç: DCA'nın iz toplamasını, DFA'nın hata enjekte etmesini **zorlaştırmak**

## Karşı önlem 4 · Anahtar yenileme

---

- Anahtar sık yenilenirse, bir anahtarı çıkarmanın **değeri düşer**
- x. hafta: **kripto-periyot** (bir anahtarın ömrü)

## Karşı önlem 5 · Cihaz/sürüm bağlama

---

- Tablolar yalnız **belirli bir cihazda/sürümde** anlamlı olur
- **Kod taşımayı** (code lifting) engeller

## Karşı önlem 6 · Sunucu tarafı risk denetimi

---

- Uçtaki koruma ne olursa olsun, işlem **sunucuda** ayrıca değerlendirilir
- Hız sınırı, anomali tespiti, sayaç/ATC denetimi
- **Son ve en güvenilir** savunma

## Karşı önlem 7 · Donanıma taşı

---

- Mümkünse anahtarı **hiç yazılıma koyma**
- TEE, güvenli öge (SE), HSM

Bu, bir sonraki bölümün konusu.

## Ana kural (tekrar)

---

WBC bir **katmandır**, çözüm değil.

Doğru ifade: "anahtar çıkarmayı şu kadar geciktiriyorum; asıl güvencem anahtar yenileme, cihaz bağlama ve sunucu risk denetiminde."

## Bölüm 3 — kısa sinama

---

1. "Hepsi kırıldı" ifadesi kararınızı nasıl etkiler?
2. DCA hangi klasik saldırıya benzer?
3. İç kodlamalar DCA'yı neden her zaman durdurmaz?
4. En güvenilir son savunma hangisi?

## Bölüm 3 — cevaplar

---

1. WBC **tek başına** güvence değil; anahtar yenileme + cihaz bağlama + sunucu denetimi + gizleme ile; mümkünse **donanım**a taşı.
2. **DPA**'ya (güç analizi) — DCA onun yazılım hâlidir.
3. İç kodlama birebir eşlemedir (bijeksiyon); istatistiksel **korelasyon** çoğu zaman hayatta kalır.
4. **Sunucu tarafı** risk denetimi (son ve en güvenilir).

## 4. Katmanlı yer, donanım, proje

## Anahtar korumak: seçenekler

---

WBC tek seçenek değil. Bir anahtarı korumanın yolları var; güç ve maliyet farklı.

Sırayla, en zayıftan en güçlüye.

## Seçenek 1 · Sabit dizide düz anahtar

---

- Anahtar: yazılımda, **açık**
- Güç: **yok**
- Ne zaman: **asla**

## Seçenek 2 · Gizlenmiş/parçalanmış anahtar

---

- Anahtar: yazılımda, gizlenmiş
- Güç: çok düşük
- Ne zaman: yalnız düşük değerli, kısa ömürlü sırlar

## Seçenek 3 · WBC + katmanlı savunma

---

- Anahtar: yazılımda, **tablolar** gömülü
- Güç: orta (geciktirir)
- Ne zaman: donanım kökü yoksa; **anahtar yenileme + sunucu denetimiyle**

## Seçenek 4 · TEE / Güvenli Öğe (SE)

---

- Anahtar: **donanımla yalıtılmış**
- Güç: yüksek
- Ne zaman: cihaz destekliyorsa **tercih edilir**

## Seçenek 5 · HSM / SoftHSM

---

- Anahtar: donanım (ya da yazılım benzetimi) modülünde
- Güç: yüksek (HSM); SoftHSM yalnız test
- Ne zaman: **sunucu tarafı**; anahtar hiç dışarı çıkmaz

## Seenekler · tek tablo

| Seenek       | Anahtar nerede   | Gü       |
|---------------|------------------|-----------|
| Düz dizi      | Yazılımda açık   | Yok       |
| Gizli/paralı | Yazılımda gizli  | ok düřük |
| WBC + katman  | Tablolara gömölü | Orta      |
| TEE / SE      | Donanım yalıtık  | Yüksek    |
| HSM / SoftHSM | Donanım modölü   | Yüksek    |

# Karar kuralı

Anahtarı yazılıma koymak **zorunda değilsen, koyma.**

- Donanım kökü (TEE/SE/HSM) varsa **onu** kullan
- WBC, donanım kökünün **olmadığı** durumda bir **ödünleşim** çözümüdür
- Her zaman: anahtar yenileme + cihaz bağlama + sunucu denetimiyle birlikte

## SoftHSM ve PKCS#11 (sunucu köprüsü)

---

- **PKCS#11**: anahtar modülleriyle konuşmanın standart arayüzü
- Uygulama anahtarın **değerini görmez**; "şunu imzala/şifrele" der
- **SoftHSM**: aynı arayüzü sunan yazılım benzetimi; gerçek donanım koruması **yok** → geliştirme/test için

WBC uçtaki (istemci) problem; bu sunucudaki anahtar problemine yanıt.

# Hatalı → saldırı → koruma

# 1 · Hatalı tasarım

---

```
/* İstemci uygulaması: veri şifreleme anahtarı gömülü */  
static const uint8_t k[16] = { ... }; /* KÖTÜ */
```

Bir istemci, bir veri şifreleme anahtarını sabit diziye gömer.

## 2 · Saldırı (kavramsal)

---

1. `strings` + **entropi taraması** → yüksek entropili 16 baytlık blok
2. Hata ayıklayıcı → bu bloğun şifreleme çağrısında kullanıldığını doğrular
3. Anahtar **dakikalar** içinde çıkar

## 3 · Koruma (katmanlı) — a

---

- Anahtarı mümkünse **TEE/SE**'ye taşı
- Değilse **WBC tablolarına** göm (düz bayt dizisi kalmasın)

## 3 · Koruma (katmanlı) — b

---

- WBC'yi 9. haftanın **gizlemesiyle** ve 6. haftanın **RASP**'ıyla sar
  - DCA için iz toplamak, DFA için hata enjekte etmek zorlaşır
- **Cihaz/sürüm bağla** → kod taşıma işe yaramaz

## 3 · Koruma (katmanlı) — c

---

- Anahtarı **sık yenile** (kısa kript-periyot)
- **Sunucuda risk denetimi** uygula (hız sınırı, anomali)

## 4 · Dürüst kalan risk

---

- Yeterince kararlı bir saldırgan **tek bir cihazda** anahtarı yine çıkarabilir
- Ama artık:
  - Bir kopyayı kırmak **bütün sistemi açmaz** (cihaz bağlama)
  - Çıkarılan anahtarın **ömrü kısa** (yenileme)
  - Sunucu, anormal kullanımı yakalar

## Senaryodan çıkan kurallar

---

- Anahtarı düz gömme
- Donanım kökü varsa kullan
- WBC'yi **asla** tek başına bırakma
- Cihaz bağlama + yenileme + sunucu denetimi

# Proje ve kapanış

## Proje · S8 (anahtar koruma gerekçesi) — 1

---

En az bir hassas anahtar için bir koruma kararı ve **gerekçesi** yazın:

1. Anahtar nerede durur, nasıl korunur? (düz değilse: TEE/SE, WBC, gizleme — hangisi, **neden**)

## Proje · S8 — 2

---

2. Koruma yeterli değilse (ör. saf yazılımdaysanız) bunu **açıkça** yazın ve telafi katmanlarını belirtin:

- anahtar yenileme periyodu
- cihaz/sürüm bağlama
- sunucu risk denetimi

## Proje · S8 — 3

---

3. WBC kullanmadıysanız **neden** kullanmadığının gerekçesi:

- maliyet
- donanım kökü var
- varlık değeri düşük

Gerekçeli bir "kullanmadık" **tam** bir yanıttır.

## Depoda gerçek anahtar olmasın

---

- Bütün anahtar dosyaları, tablolar ve örnekler **sentetik** olmalı
- Sertifikasyonda depoda gerçek anahtar = en ağır bulgulardan biri
- Projede de öyle sayılır

## Terimler sözlüğü (1)

| Terim               | Anlam                                       |
|---------------------|---------------------------------------------|
| Kara/gri/beyaz kutu | Saldırganın gördüğü                         |
| WBC                 | Beyaz kutuda çalışan kript                  |
| S-box               | Baytı değiştiren sabit tablo                |
| Bijeksiyon          | Tersi alınabilen birebir eşleme             |
| İç/dış kodlama      | Tablo içi / tüm şifreyi saran gizli dönüşüm |

## Terimler sözlüğü (2)

| Terim          | Anlam                                   |
|----------------|-----------------------------------------|
| Kod taşıma     | Anahtarı çıkarmadan kodu kopyalama      |
| DCA            | Çalışma izlerine DPA benzeri istatistik |
| DFA            | Hata enjekte edip anahtarı hesaplama    |
| Kripto-periyot | Bir anahtarın ömrü                      |
| TEE/SE/HSM     | Donanım tabanlı anahtar koruması        |

## Kendini sinama (1–6)

---

1. Üç saldırgan modeli; AES kanıtı hangisini varsayar?
2. "Tek turu çalıştır" ve "hata enjekte et" hangi saldırılara zemin?
3. Diziye gömülü anahtar neden koruma değil? (entropi)
4. Kod taşıma nedir, hangi önlem kapatır?
5.  $T[x] = S\text{-box}[x^k]$  tek başına neden güvensiz?
6. İç ve dış kodlama farkı; dış kodlamanın sınırı?

## Kendini sına — cevaplar (1–6)

1. **Kara kutu** (yalnız G/Ç), **gri kutu** (+yan kanal: güç/zaman), **beyaz kutu** (tam erişim). Klasik AES kanıtı **kara kutuyu** varsayar.
2. "Tek turu çalıştır/ara değeri gözle" → **DCA**; "hata enjekte et" → **DFA**.
3. Anahtar **yüksek entropili** bir blok olarak durur → entropi taraması/ **strings** /desenle bulunur (demo **--tara** ). Konum değiştirmek gizleme değil.
4. **Code lifting**: tabloyu/rutini olduğu gibi kopyalayıp başka yerde kullanmak. Kapatan: **dış kodlama** (+ cihaz bağlama/sunucu denetimi).
5.  $T[x] = S\text{-box}[x^k]$  doğrudan  $k$ 'ye bağlı; tablo S-box ile karşılaştırılıp  **$k$  geri hesaplanır**. **İç kodlamayla** maskele.
6. **İç kodlama** ara değerleri gizli bijeksiyonlarla maskeler; **dış kodlama** giriş/çıkışı dönüştürür (code lifting'i zorlaştırır). Sınır: **çağırının da eşleşmesi** gerekir → standart AES uyumu bozulur.

## Kendini sınaama (7–12)

---

7. Tablo boyutu/hız mertebesi? Neden seçili işlemlere?
8. "Hepsi kırıldı" kararınızı nasıl etkiler?
9. DCA hangi donanım saldırısına benzer?
10. Anahtar koruma seçeneklerini güç/maliyetle sırala
11. WBC kullanıyorsan eklemen gereken 3 katman?
12. Bir anahtar için S8 gerekçesini üç cümleyle yaz

## Kendini sinama — cevaplar (7–12)

---

7. Tablolar **MB mertebesinde** ve kara kutuya göre onlarca-yüzlerce kat **yavaş** → yalnız **seçili/kritik** işlemlere uygulanır.
8. WBC'yi tek başına anahtar koruması saymayız; **geciktirici katman** olarak yenileme + bağlama + sunucu denetimiyle kullanırız (mümkünse donanım).
9. **DPA'ya** (Differential Power Analysis); DCA güç izinin yerine **yazılım izini** (bellek erişimi/ara değer) koyar.
10. (zayıf→güçlü) düz gömülü anahtar < kodlanmış tablo (saf-yazılım WBC) < WBC+bağlama/yenileme < **TEE/SE** < **HSM/donanım**. Maliyet aynı yönde artar.
11. **Anahtar yenileme · cihaz/uygulama bağlama · sunucu tarafı denetim/iptal** (+ RASP/bütünlük).
12. Örnek: "DEK, beklemedeki veriyi AES-256-GCM ile korur. HSM'de üretilir/saklanır, düz bellekte tutulmaz. Saf-yazılım whitebox'lar kırıldığı ve varlık değeri yüksek olduğu için **donanım** seçildi."

## Özet: bu haftanın tek cümlesi

---

Yayımlanmış saf-yazılım whitebox'ların hepsi kırıldı; WBC anahtar çıkarmayı **geciktiren bir katmandır** ve yenileme, cihaz bağlama, sunucu denetimiyle birlikte anlam kazanır.

# Kaynaklar

---

- **Güvenli programlama teknik kılavuzu** — WBC saldırgan modeli, whitebox AES'in konumu
- Chow, Eisen, Johnson, van Oorschot — WB-AES (SAC 2002), WB-DES (DRM 2002)
- Wyseur — "Hiding Keys in Software" (kısa giriş)
- BGE (SAC 2004); Bos vd. DCA (CHES 2016); "Unboxing the White-Box" (BH EU 2015)
- WhibOx yarışması sonuçları (2017–2024)

# Sonraki hafta

12. hafta — Sertifikasyon ve sızma testi planlaması

Bir korumanın "ne kadar dayandığını" ölçmek → bağımsız değerlendirme süreci ve raporlama.

# Ek A · Minik sayısal örnek

## Neden oyuncak örnek?

---

Gerçek AES 256 girişli tablolarla çalışır; tahtaya sığmaz.

Fikri **4 değerli** minik bir örnekle göreceğiz.

Amaç: "anahtarı tabloya pişirmek" ne demek, **elle** görmek.

## Minik S-box tanımı

Diyelim 2 bitlik değerlerimiz var (0,1,2,3) ve şu sabit S-box:

|           |   |   |   |   |
|-----------|---|---|---|---|
| x:        | 0 | 1 | 2 | 3 |
| S-box[x]: | 3 | 2 | 0 | 1 |

Bu tablo **herkese açık** (algoritmanın parçası, sır değil).

# Anahtar ve işlem

---

- Anahtar  $k = 1$  (gizli).
- İşlem (bir AES turunun mini hâli):  $\text{ç1kt1} = \text{S-box}[x \wedge k]$ .

$\wedge$  = XOR.  $x \wedge 1 : 0 \leftrightarrow 1, 2 \leftrightarrow 3$  (son biti çevirir).

## Adım adım: normal işlem

```
x=0 → x^1=1 → S-box[1]=2  
x=1 → x^1=0 → S-box[0]=3  
x=2 → x^1=3 → S-box[3]=1  
x=3 → x^1=2 → S-box[2]=0
```

Burada `k=1` kodda **açıkça** kullanılıyor. Saldırgan görür.

## Anahtarı tabloya "pişir"

$T[x] = S\text{-box}[x \wedge 1]$  diye **önceden** hesaplayıp tablo yapalım:

|       |   |   |   |   |
|-------|---|---|---|---|
| x:    | 0 | 1 | 2 | 3 |
| T[x]: | 2 | 3 | 1 | 0 |

Artık kodda **k** **yok**; sadece **T** var. Görünüşte anahtar kayboldu.

## Ama anahtar sızıyor! (kodlama yoksa)

Saldırgan  $T$  'yi bilinen  $S\text{-box}$  ile karşılaştırır:

```
S-box: 3 2 0 1  
T:      2 3 1 0
```

$T[x] = S\text{-box}[x \wedge k]$  olduğunu bilerek hangi  $k$  uyuyor diye dener:  $k=1$  uyar.

**Sonuç:** kodlanmamış tablo anahtarı ele verir. İşte bu yüzden iç/dış kodlama var.

## Kodlama fikri (minik)

---

Çıkışa gizli bir eşleme  $E$  uygula:  $T'[x] = E(T[x])$ .

$E$ :  $0 \rightarrow 1, 1 \rightarrow 3, 2 \rightarrow 0, 3 \rightarrow 2$  (gizli bijeksiyon)

$T'$ :  $E(2) E(3) E(1) E(0) = 0 2 3 1$

Artık  $T'$ ,  $S$ -box'a benzemez; basit karşılaştırma  $k$ 'yi vermez.

## Kodlamanın bedeli

---

- Bir sonraki adım **E** 'nin **tersini** uygulamalı ki sonuç doğru çıksın.
- Gerçek AES'te bu, tabloları **zincirleyerek** yapılır (iç kodlamalar).
- Her kodlama fazladan tablo ve boyut demektir → **maliyet**.

Minik örnek bile fikri gösteriyor: gizlilik ucuz değil.

## Minik örnekten çıkan ders

---

1. Anahtarı tabloya gömmek onu **görünmez** yapmaz (kodlama şart)
2. Kodlama işi çözer ama **boyut/karmaşıklık** ekler
3. Gerçekte tablolar yüzlerce KB olur

Bu, bölüm 2'deki beş adımın **sezgisidir**.

## Ek B · DCA neden çalışır? (sezgi)

## Sezgi: neyi izliyor?

---

- WBC çalışırken tablolardan **ara değerler** okur.
- Bu ara değerler, gizli anahtara **bağlı** değişir.
- DCA bu değerlerin izini toplar.

## İstatistik nasıl anahtarı verir?

---

- Saldırgan bir anahtar baytı için **tahmin** yapar.
- Tahmine göre ara değerin nasıl davranması gerektiğini hesaplar.
- Toplanan izlerle **en iyi uyan** tahmin, doğru bayttır.

Bu, DPA'nın (güç analizi) **yazılım** hâlidir.

## İç kodlama neden DCA'yı durdurmaz?

---

- İç kodlama ara değeri **karıştırır** ama **birebir** eşlemedir (bijeksiyon)
- İstatistiksel korelasyon çoğu zaman **hayatta kalır**
- Bu yüzden 2016'dan sonra WBC'ler DCA'ya karşı savunmasız kaldı

## Karşı önlem sezgisi

---

- **Doğrusal olmayan maskeleme:** korelasyonu bozacak biçimde ara değerleri gizler
- **Dış kodlama:** saldırganın gördüğü giriş/çıkışı standart olmaktan çıkarır
- **RASP:** iz toplamayı (araç bağlamayı) zorlaştırır

Hiçbiri kesin çözüm değil; hepsi **maliyet + gecikme**.

# Ek C · Karar akışı

# "Bu anahtarı nasıl korurum?" — 1

---

**Soru 1:** Cihazda TEE/güvenli öge var mı?

- **Evet** → anahtarı oraya koy. **Bitti** (en güçlü).
- **Hayır** → devam.

## "Bu anahtarı nasıl korurum?" — 2

---

**Soru 2:** Bu sunucu tarafı bir anahtar mı?

- **Evet** → HSM/PKCS#11 (üretimde HSM, testte SoftHSM).
- **Hayır (istemci, donanım yok)** → devam.

## "Bu anahtarı nasıl korurum?" — 3

---

**Soru 3:** Varlık değeri koruma maliyetini haklı çıkarıyor mu?

- **Düşük** → hafif gizleme + kısa ömür yeter.
- **Yüksek** → WBC + katmanlı savunma + yenileme + cihaz bağlama + sunucu denetimi.

## "Bu anahtarı nasıl korurum?" — 4

---

Her durumda:

- Anahtarı **düz gömme**
- Kararı ve gerekçesini **S8'e yaz**
- Kalan riski açıkça belirt

## Karar akışı · tek bakış

```
TEE/SE var mı? --evet--> donanıma koy (bitti)
| hayır
Sunucu anahtarı mı? --evet--> HSM/PKCS#11
| hayır
Varlık değeri yüksek mi?
    düşük -> hafif gizleme + kısa ömür
    yüksek -> WBC + katman + yenileme + cihaz bağlama + sunucu
```

## Ek D · Mini vaka (sentetik)

## Vaka · kurgu

---

Bir mobil uygulama, sunucudan indirdiği **oturum anahtarıyla** yerel veriyi şifreliyor.

- Cihazların bir kısmında TEE var, bir kısmında yok.
- Anahtar günde bir yenileniyor.

Nasıl koruruz?

## Vaka · TEE olan cihazlar

---

- Anahtarı **TEE'ye** koy.
- Uygulama anahtarı hiç görmez; TEE içinde şifreleme yapılır.
- En güçlü çözüm; ek WBC gereksiz.

## Vaka · TEE olmayan cihazlar

---

- Anahtarı **WBC tablolarına** göm (düz dizi yok).
- ix. hafta gizleme + 6. hafta RASP ile sar.
- **Cihaz bağla:** tablolar başka cihazda çalışmasın.

## Vaka · ortak katmanlar

---

- Anahtar **günde bir yenileniyor** → çıkarılanın ömrü kısa.
- **Sunucu**, anormal kullanımı (çok istek, tuhaf konum) yakalıyor.
- Böylece bir cihaz kırılrsa bile hasar sınırlı.

## Vaka · kalan risk (dürüst)

---

- TEE'siz cihazda kararlı saldırgan anahtarı çıkarabilir.
- Ama: tek cihazla sınırlı + 24 saat ömür + sunucu denetimi.
- Bu ödünleşim **S8'de açıkça yazılır.**

## Vaka · çıkarım

---

Tek bir "sihirli" koruma yok.

Güç, **katmanların birleşiminden** ve **dürüst kalan risk analizinden** geliyor.

Değerlendirici (12. hafta) tam da bunu arıyor.

## Ek — kapanış notu

---

Bu ekler sınavda ayrıca sorulmaz; ama:

- Minik örnek "pişirme + kodlama" sezgisini verdi
- DCA sezgisi "neden hepsi kırıldı"yı açıkladı
- Karar akışı + vaka, S8'i yazmanıza yardım eder

WBC'yi **anlamak**, onu **doğru yere koymak** demektir.

# Ek E · Çözümlü kendini sinama

## Soru 1

---

Kara, gri ve beyaz kutu modellerini bir cümleyle ayırın. AES kanıtı hangisini varsayar?

## Cevap 1

---

- **Kara:** yalnız girdi/çıktı.
- **Gri:** + yan kanallar (zaman, güç).
- **Beyaz:** her şey + değiştirebilir.

AES'in güvenlik kanıtı **kara kutu** varsayar; teslimde biz beyaz kutudayız.

## Soru 2

---

**WBC saldırganının "tek turu çalıştırma" ve "hata enjekte etme" yetenekleri hangi saldırılara zemin hazırlar?**

## Cevap 2

---

- Tek turu çalıştırma + iz toplama → **DCA**.
- Hata enjekte etme → **DFA**.

İkisi de tasarımın iç matematiğini bilmeden çalışır.

## Soru 3

---

Anahtarı sabit diziye gömmek neden koruma değildir? Entropi taraması burada ne işe yarar?

## Cevap 3

---

- Program çalışırken anahtarı kullanır; blok bellekte/ikili dosyada durur.
- **Entropi taraması** yüksek entropili (rastgele görünen) bloğu bulur → "anahtar burada".
- `strings` + gdb ile dakikalar içinde doğrulanır.

## Soru 4

---

Kod taşıma (code lifting) nedir? Hangi önlem kapatır?

## Cevap 4

---

- Saldırgan anahtarı **çıkarmadan**, şifreleme yapan kodu (tablolar + yorumlayıcı) kopyalar.
- İşlevi anahtarı bilmeden çalar.
- Önlem: **cihaz/sürüm bağlama** — tablolar yalnız o cihazda anlamlı.

## Soru 5

---

$T[x] = \text{S-box}[x^k]$  tek başına neden güvensiz?

## Cevap 5

---

- $T$ , bilinen  $S\text{-box}$ 'ın  $x \wedge k$  ile ötelenmiştir.
- Saldırgan iki tabloyu karşılaştırıp öteleme miktarından  $k$ 'yı çıkarır.
- Bu yüzden iç/dış **kodlama** gerekir.

## Soru 6

---

İç kodlama ile dış kodlama (F, G) farkı? Dış kodlamanın en büyük sınırı?

## Cevap 6

---

- **İç:** tablolar arası, ara değerleri karıştırır.
- **Dış (F, G):** tüm şifreyi sarar (  $G \circ AES \circ F^{-1}$  ).
- **Sınır:** artık **standart AES değil**; karşı taraf F/G'yi bilmeli → EMV gibi standartlarda kullanılamayabilir.

## Soru 7

---

Yaklaşık tablo boyutu ve hız maliyeti? Neden yalnız seçili işlemlerde?

## Cevap 7

---

- Tablo: **yüzlerce KB** (std AES anahtarı 16–32 bayt).
- Hız: **~50× yavaş**.
- Bu maliyet, WBC'yi yalnız **küçük ve kritik** işlemlere sınırlar.

## Soru 8

---

"Yayımlanmış bütün saf-yazılım WBC kırıldı" ifadesi projedeki kararınızı nasıl etkiler?

## Cevap 8

---

- WBC'yi **tek başına** anahtar güvencesi sayamazsınız.
- Kullanırsanız: yenileme + cihaz bağlama + sunucu denetimi + gizleme/RASP ile.
- Mümkünse donanıma (TEE/SE) taşıyın.

## Soru 9

---

**DCA klasik hangi donanım saldırısına benzer? İç kodlamalar onu neden her zaman durdurmaz?**

## Cevap 9

---

- **DPA**'ya (güç analizi) benzer — yazılım hâli.
- İç kodlama ara değeri karıştırır ama birebir eşlemedir; istatistiksel korelasyon çoğu zaman hayatta kalır.

## Soru 10

---

Anahtar koruma seçeneklerini güç/maliyet ekseninde sıralayın.

## Cevap 10

---

Düz dizi (yok) → gizli/parçalı (çok düşük) → **WBC + katman** (orta) → **TEE/SE** (yüksek) → **HSM** (yüksek, sunucu).

Kural: yazılıma koymak zorunda değilsen koyma.

## Soru 11

---

WBC kullanıyorsanız, tek başına bırakmamak için en az üç katman?

## Cevap 11

---

1. Gizleme (9. hafta) + RASP (6. hafta)
2. Anahtar yenileme (kısa kript-periyot)
3. Cihaz/sürüm bağlama + sunucu risk denetimi

## Soru 12

---

Projenizdeki bir anahtar için S8 gerekçesini üç cümleyle yazın (örnek).

## Cevap 12 (örnek)

"Oturum anahtarı TEE'siz cihazlarda WBC tablolarında tutulur ve gizleme+RASP ile sarılır. Kod taşımaya karşı  
cihaza bağlanır; anahtar 24 saatte bir yenilenir. Kalan risk: tek cihazda çıkarılabilir, ama sunucu  
risk  
denetimi ve kısa ömür hasarı sınırlar."

## Kapanış · üç hafta bir arada

---

- **9. hafta:** gizleme kuralları (kod)
- **11. hafta:** whitebox (anahtar) — bugün
- **14. hafta:** otomasyon (Tigress)

Ortak kural: koruma **kırılamazlık değil, gecikme**; güç katmanlardan ve ölçümden gelir.